

Functions

Warmup activity

Work on the activity (handout) with a neighbor, then we will discuss as a class

Warmup

In R: arithmetic with vectors is entrywise

E.g. $y \leftarrow c(0, 1, 2)$

$y + 1 \rightarrow (1, 2, 3)$

```
1 z_score <- function(x) {  
2   (x - mean(x, na.rm = TRUE)) / sd(x, na.rm = TRUE)  
3 }  
4  
5 diamonds_new <- diamonds |>  
6   mutate(carat_z = z_score(carat))
```

vectorization:
applying a function
to a vector
does that function
for each entry

What does this code do?

get a z-score for each entry in carat column

calculated as

$$\frac{\text{value} - \text{mean(carat)}}{\text{sd(carat)}}$$

← mean of all values in column

carat - z

sd of all values in column

Warmup

```
1 z_score <- function(x) {  
2   (x - mean(x, na.rm = TRUE)) / sd(x, na.rm = TRUE)  
3 }  
4  
5 diamonds_new <- diamonds |>  
6   mutate(carat_z = z_score(carat))
```

The diamonds dataset has 53940 rows and 10 columns.
What will be the dimensions of the diamonds_new dataset?

53940 rows and 11 columns

Functions

name of function
arguments to function (input)

```
1 z_score <- function(x) {  
2   (x - mean(x, na.rm = TRUE)) / sd(x, na.rm = TRUE)  
3 }
```

body
(what function does)

R has implicit return:

output of last line will get returned if it would
get printed to console if we were running
code outside function

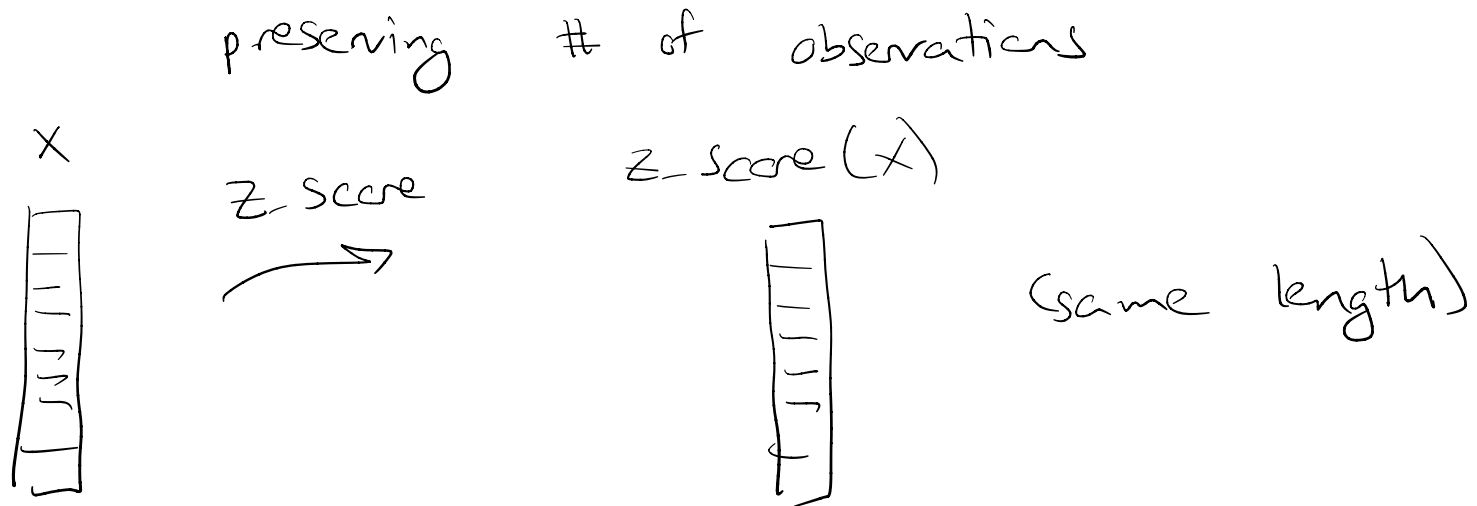
can also use explicit return:

```
z_score <- function(x) {  
  return((x - mean(x, ...)) / sd(x, ...))  
}
```

Functions

```
1 z_score <- function(x) {  
2   (x - mean(x, na.rm = TRUE)) / sd(x, na.rm = TRUE)  
3 }  
4  
5 diamonds_new <- diamonds |>  
6   mutate(carat_z = z_score(carat))
```

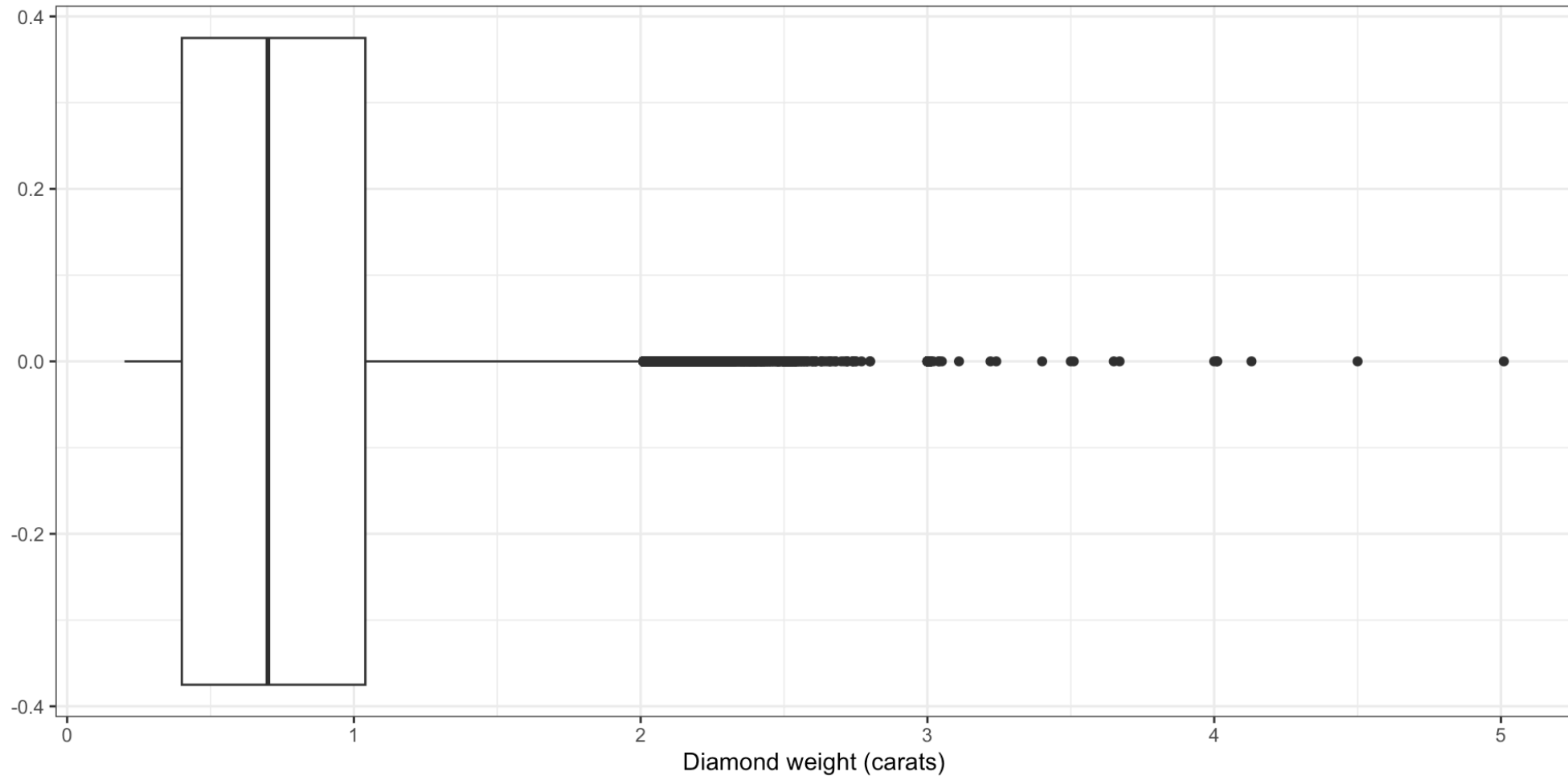
R for Data Science calls the `z_score` function a “mutate” function.



Why are functions nice?

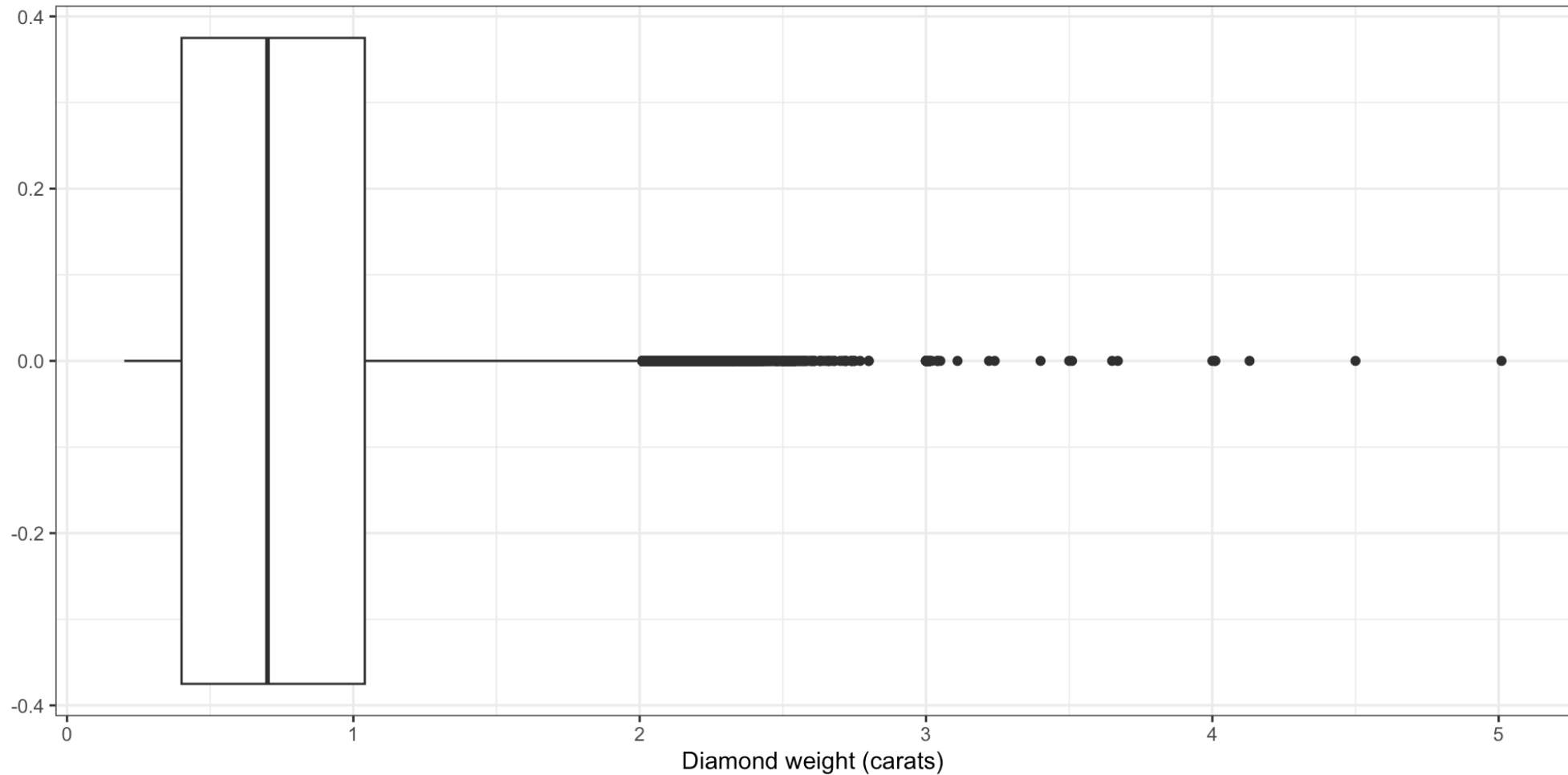
- Implement new functions to apply to datasets (transformations, summary statistics, etc.)
- Functions make our code easier to read
 - If we need to do the same process multiple times, create a function!
 - Example: `clean_ggbo` function on current HW
- Functions can be tested to ensure they behave the way we want
 - Unit testing (upcoming topic)

Another challenge



What are the individual points on the right of the boxplot?

Another challenge: identifying outliers



How do we identify outliers when constructing a boxplot?

points outside

$(Q1 - 1.5 IQR, Q3 + 1.5 IQR)$

Identifying outliers

We wish to write a function that we can use to identify outliers in numeric variables.

What should we name the function?

Something informative, e.g. find_outliers
is_outliers
:

Identifying outliers

We wish to write a function that we can use to identify outliers in numeric variables.

What should the input to the function be?

`find_outliers <- function (x) {`

*numeric vector
(e.g. column in
a dataframe)*

`}`

Identifying outliers

```
1 find_outliers <- function(x) {  
2  
3 }
```

What needs to happen inside the function?

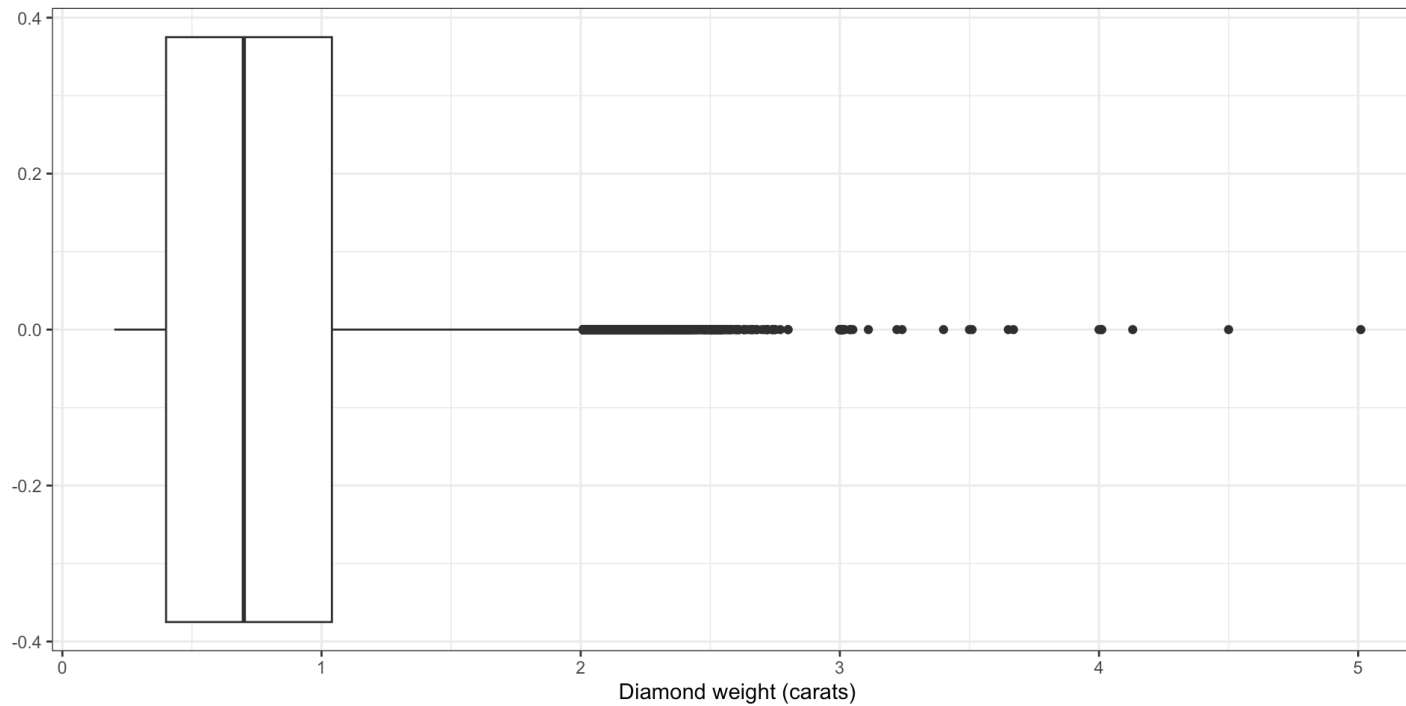
- Compute Q_1 & Q_3
- IQR of x : $Q_3 - Q_1$
- calculate lower & upper bounds
 $Q_1 - 1.5 \text{ IQR}$
 $Q_3 + 1.5 \text{ IQR}$
- see which values are outside that range
- return a vector (same length as x)
where each entry is T or F

Identifying outliers

```
1 find_outliers <- function(x) {  
2  
3 }
```

(Switch to R Studio)

Identifying outliers



```
1 diamonds |>  
2   mutate(carat_outliers = find_outliers(carat)) |>  
3   filter(carat_outliers) |>  
4   pull(carat) |>  
5   head()
```

```
[1] 2.06 2.14 2.15 2.22 2.01 2.01
```

Counting outliers

```
1 find_outliers <- function(x) {  
2   q1 <- quantile(x, 0.25)  
3   q3 <- quantile(x, 0.75)  
4   iqr <- q3 - q1  
5   (x > q3 + 1.5*iqr) | (x < q1 - 1.5*iqr)  
6 }
```

What if we want to *count* the number of outliers?

. T T F F T

T + T → 2

T + F → 1

F + F → 0

(T == "1" , F == "0")

sum(find_outliers(x))

↳ # of TRUEs
 (# of outliers)

Counting outliers

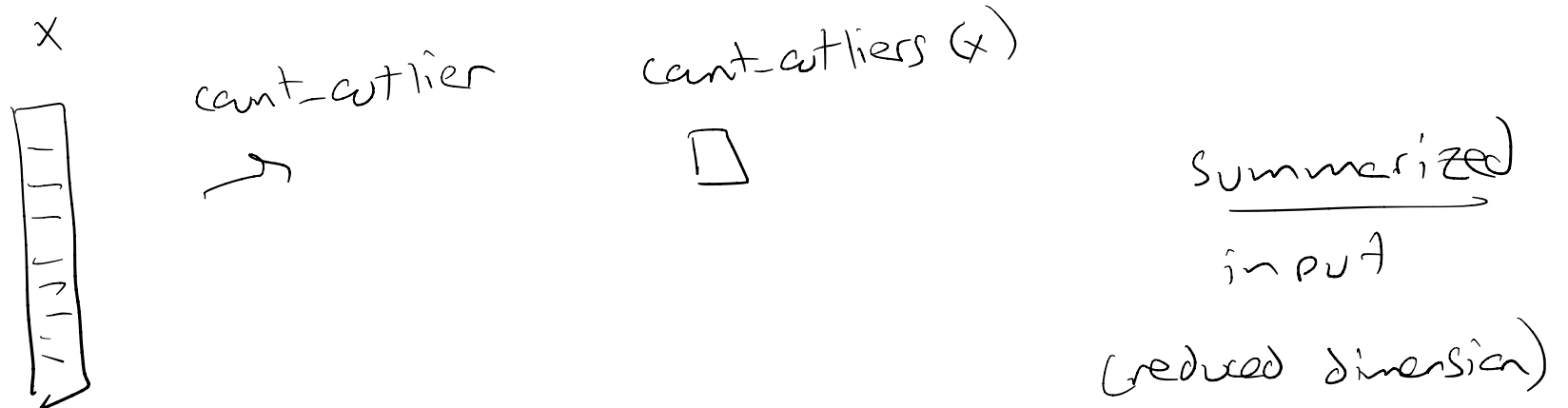
```
1 find_outliers <- function(x) {  
2   q1 <- quantile(x, 0.25)  
3   q3 <- quantile(x, 0.75)  
4   iqr <- q3 - q1  
5   (x > q3 + 1.5*iqr) | (x < q1 - 1.5*iqr)  
6 }  
7  
8 count_outliers <- function(x) {  
9   sum(find_outliers(x))  
10 }
```


Counting outliers

```
1 diamonds |>
2   summarize(carat_outliers = count_outliers(carat))
```

```
# A tibble: 1 × 1
  carat_outliers
      <int>
1           1889
```

R for Data Science would call the `count_outliers` function a “summarize” function. Why?



Counting outliers

```
1 diamonds |>  
2   summarize(carat_outliers = count_outliers(carat))
```

```
# A tibble: 1 × 1  
  carat_outliers  
      <int>  
1           1889
```

What if I want to count outliers for multiple variables?

Counting outliers

```
1 diamonds |>
2   summarize(across(c(carat, depth, price),
3                     list("outliers" = count_outliers)))
```

A tibble: 1 × 3

	carat_outliers	depth_outliers	price_outliers
	<int>	<int>	<int>
1	1889	2545	3540

Class activity

- Work with a neighbor on the class activity
- At the end of class, submit your work as an HTML file on Canvas (one per group, list all your names)

For next time, read:

- Chapter 25.2 in *R for Data Science*